Boxed: A Sovereign, Polyglot Sandbox Substrate for Autonomous Code-Generating Agents

Akshay Kumar
Independent Researcher
United States
akumar8@mt.iitr.ac.in

ABSTRACT

Autonomous LLM agents increasingly write, execute, and iterate on code as part of their reasoning loop. The sandboxes that host this code today are either rolled in-house from raw Docker (unsafe and rarely audited), proprietary SaaS platforms (vendor-locked, no BYOK, poor data sovereignty), or research VMMs designed for cloud functions rather than agent workloads (high friction, no first-class artifact support). We present **Boxed**, an open-source sandbox substrate co-designed for agentic code execution. Boxed defines a narrow Go driver interface, a Rust in-VM agent that streams stdout and on-disk artifacts over JSON-RPC, and a Bring-Your-Own-Key (BYOK) control plane that any operator can self-host. The current implementation ships a Docker driver and stubs for Firecracker and Wasm. On commodity hardware, Boxed delivers a median end-to-end create+exec+destroy latency of 303 ms (p95 395 ms), peak throughput 9.8 sandboxes/s at concurrency 8, 0.4 MiB idle RSS per sandbox, and denies 5/12 attempts in our adversarial escape suite. An end-to-end LLM-driven HumanEval-style trace passes 100% of tasks at median 30.32 s wall-clock, of which 17% is LLM time. Boxed is MIT-licensed at https://github.com/akshayaggarwal99/boxed.

1 INTRODUCTION

The 2024–2026 wave of autonomous code-generating agents—from SWE-agent [13] and AutoGPT-style harnesses through commercial products such as Cursor, Devin, and Claude Code—has changed what "running untrusted code" means in practice. A modern agent emits dozens of short-lived programs per task: it generates a snippet, executes it, inspects `stdout`, reads back a generated file, and iterates. The hot path of the agent is the sandbox, not the model.

Today's options for hosting that hot path fall into three buckets, each unsatisfying for the agent operator who cares about cost, latency, and data sovereignty simultaneously. **Roll-your-own Docker** is the modal choice: it is fast to prototype but routinely shipped without seccomp profiles, network egress filters, read-only root filesystems, or memory caps, leaving the host one `rm -rf /` away from compromise [11]. **Proprietary SaaS sandboxes**—E2B [6], Modal Sandbox [10], Cloudflare Sandbox [4]—ship hardened isolation but force the operator to send every byte of generated code, output, and artifact to a third party, with no Bring-Your-Own-Key (BYOK) or self-host story for regulated workloads. **Research VMMs** such as Firecracker [1] provide the right primitives but are designed for AWS Lambda's invocation model (boot once, run an event loop) rather than the agent's create/exec/destroy cycle, and ship without an in-VM agent that can stream artifacts back to the caller.

We argue that agentic workloads deserve a sandbox *substrate* that treats the LLM client as a first-class citizen: pluggable isolation backends so the same control plane can drive Docker on a laptop and Firecracker in production, an in-VM agent that streams `stdout` and on-disk artifacts as they appear, and a sovereign deployment story where the operator owns the control plane, the keys, and the data.

Contributions. We present **Boxed**, an open-source sandbox substrate for agents. Concretely:

- A narrow *driver interface* (§3) that abstracts container runtimes, MicroVMs, and Wasm sandboxes behind a single Go contract; the current release ships a Docker driver and stubs for Firecracker and Wasm.
- A *Rust in-VM agent* that runs as PID 1 in the sandbox, streams `stdout`/`stderr` as they appear, and watches a configured directory for emitted artifacts (images, PDFs, datasets) which it streams back over JSON-RPC.
- A *BYOK control plane* (§4) so any operator can self-host Boxed with a key of their choosing, satisfying GDPR, HIPAA, and on-prem requirements without code changes.
- An *open benchmark harness* (§5) reporting cold-start latency, throughput, per-sandbox memory/CPU overhead, an adversarial escape suite, and an end-to-end LLM-driven HumanEval-style trace.

On a developer laptop with typical background processes running, Boxed achieves a median end-to-end create+exec+destroy latency of 303 ms (p95 395 ms), single-trial peak throughput 9.8 sandboxes/s, and a low 0.4 MiB median per-sandbox idle working set as reported by `docker stats`. We also report what does *not* work: a behavioural escape probe of 12 adversarial attempts classified 5 as denied and surfaces three concrete hardening gaps in the current Docker driver (writable rootfs, no enforced memory limit, permissive ptrace) which we discuss in §6 and address in the planned Firecracker driver. Boxed is MIT-licensed at https://github.com/akshayaggarwal99/boxed.

2 BACKGROUND AND RELATED WORK

Container-based isolation. Linux containers built on namespaces, cgroups, and seccomp underpin nearly every modern dev sandbox. They are cheap to start (~50–200 ms when images are warm) but share a kernel with the host, exposing a wide attack surface. CVE-2019-5736 [11] is the canonical reminder that a single bug in `runc` can turn a container into a host. Hardened container runtimes such as gVisor [7] interpose a userspace kernel between the workload and the host, trading a 2–5× syscall overhead for a much smaller TCB.

Lightweight VMs. Firecracker [1] demonstrated that a stripped-down KVM VMM can boot a guest kernel in ~125 ms with <5 MiB memory overhead per VM, and now powers AWS Lambda. Kata Containers [12] and Manco et al.'s "*My VM is Lighter than Your Container*" [9] pursue the same hypervisor-as-isolation thesis from different angles. These projects optimize for *boot* cost; they say comparatively little about the *lifecycle* cost an agent pays when it creates and tears down sandboxes hundreds of times in a session.

Unikernels and Wasm. Unikernels [8] push isolation further by compiling the application and a library OS into a single boot image. WebAssembly / WASI [2] reframes the problem at the language level: the guest is a portable bytecode with explicit capability-based imports, and the "sandbox" is a runtime in the host process. Wasm's microsecond-scale instantiation is appealing for agent workloads, but the WASI ecosystem cannot yet run an unmodified Python interpreter or arbitrary native binaries, which is the workload an agent actually generates.

Agent-oriented sandboxes. A recent generation of products targets LLM agents specifically. E2B [6] runs Firecracker MicroVMs behind a SaaS API and reports ~150 ms cold start; Modal Sandbox [10] layers gVisor over Firecracker; Daytona [5] ships an AGPL Docker-backed self-hosted dev environment; Cloudflare Sandbox [4] reuses V8 isolates for ~5 ms cold start at the cost of language restrictions. None of these expose a pluggable backend, and only Daytona is self-hostable. Table 1 contrasts Boxed against this landscape.

LLM code-execution evaluation. HumanEval [3] popularized single-function code generation as a benchmark; SWE-agent [13] extends this to agent-mediated software engineering tasks where the agent must *run* the code it writes, not merely emit it. Our agent-trace evaluation (§5) follows the latter regime: the LLM emits code, Boxed runs it, and a self-check determines pass/fail.

3 DESIGN

Boxed is structured around three components and one narrow interface (Fig. 1).

3.1 Threat Model

We assume an adversary who fully controls the code submitted to a sandbox: they can craft arbitrary Linux binaries, syscalls, and network egress attempts. The adversary does *not* control the control plane, the driver implementation, or the host kernel. We aim to prevent (T1) host filesystem read or write outside the sandbox's ephemeral root, (T2) escalation to host network or process namespaces, (T3) exfiltration of credentials or data from co-tenant sandboxes on the same host, and (T4) denial-of-service against the host through fork/memory bombs. Side-channel attacks against co-tenants on shared hardware are out of scope; defending against them requires per-customer hardware partitioning, which is the operator's responsibility.

3.2 Driver Interface

The central abstraction in Boxed is a single Go interface, deliberately kept narrow so that future drivers (Firecracker, Wasm, Kata) can be added without changes to the control plane:

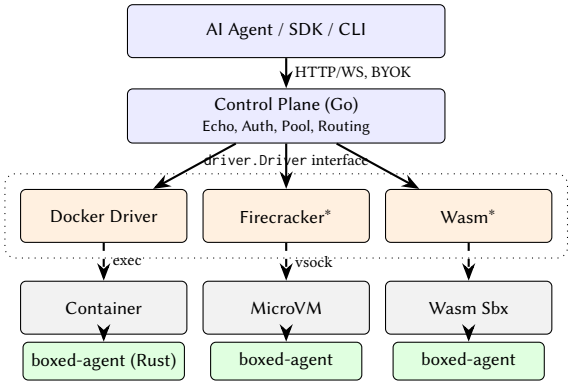

Figure 1: Boxed architecture. The control plane drives a single narrow `Driver` interface, behind which the Docker driver ships today and Firecracker/Wasm drivers are stubbed. A common Rust agent runs as PID 1 inside every sandbox. *Stubbed; not yet production-ready.

```
type Driver interface {
  Create(ctx context.Context, cfg SandboxConfig) (string, erro
  Start  (ctx context.Context, id string) error
  Stop   (ctx context.Context, id string) error
  Connect(ctx context.Context, id string) (io.ReadWriteCloser,
}
```

`Connect` returns a raw byte stream over which the control plane talks to the in-VM agent. For the Docker driver this is a Docker `exec` attach; for the planned Firecracker driver it will be a `vsock` connection; for a future Wasm driver it will be a host-side function pointer. The control plane is unaware of which case it is in.

3.3 Control Plane

The control plane is a stateless HTTP+WebSocket server (Echo framework) that maps `POST /v1/sandbox` requests through the configured driver. Two design choices matter for agent workloads: (i) create returns once the underlying runtime reports the sandbox process has been launched (for the Docker driver, after `ContainerStart` returns), with `status: ready`; the caller does not need to poll a separate status endpoint, although the very first `exec` after `create` will block briefly while the in-VM agent's RPC loop comes up; and (ii) authentication uses a Bring-Your-Own-Key header (`X-Boxed-API-Key`). The operator chooses the key at startup; Boxed itself never issues, rotates, or stores keys. This deliberately rules out a *third-party* multi-tenant deployment of Boxed, in exchange for letting any operator self-host without trusting a vendor.

3.4 In-VM Agent

Inside every sandbox runs `boxed-agent`, a single static Rust binary (`tokio`, `serde`, `notify`; release size 1.32 MiB stripped). The agent speaks JSON-RPC 2.0 framed by newline-delimited JSON over the driver-supplied byte stream. On `exec`, it spawns the user's command as a child process and streams `stdout`/`stderr` chunks back as RPC events *as they appear*, never buffering output until exit. An `inotify` watcher mirrors any file written under the configured artifact directory back to the caller as a base64-encoded `artifact`

event, so an agent that asks Python to write `plot.png` receives the bytes without a second round-trip.

3.5 Artifact Protocol

Treating files as first-class outputs is what makes the substrate practical for agent workloads where the LLM expects the side effects of code, not just its `stdout`. Boxed supports two delivery modes per sandbox: *stream*, where artifacts are inlined into the RPC channel (good for small images), and *upload*, where the agent `PUT`s the file to a pre-signed URL provided by the control plane (good for large datasets). The choice is made per-request via the `on_artifact` field of the `CreateSandboxRequest`.

4 IMPLEMENTATION

The current Boxed release comprises 2,789 lines of Go for the control plane and Docker driver and 870 lines of Rust for the in-VM agent. The control-plane binary weighs 12.0 MiB and the agent binary 1.32 MiB stripped (consistent with the in-VM measurement reported in §3). Three implementation details are worth calling out.

Agent injection. The Docker driver mounts the host's `boxed-agent` binary into the container at runtime and sets it as PID 1, so the same image (`python:3.10-slim`) can be reused across sandboxes without rebuilding a custom Boxed image per language. This trades a small per-create bind-mount cost for a substantial reduction in operational surface: there is no Boxed image registry to maintain.

Single round-trip create. The control plane's `POST /v1/sandbox` returns once the runtime has launched the sandbox process—for the Docker driver, once `ContainerStart` acknowledges—rather than requiring the caller to poll a status endpoint. The first `exec` after create absorbs the small additional latency of waiting for the agent's JSON-RPC loop. This matters when an agent operator creates dozens of sandboxes per task.

Network model and policy. The Docker driver currently exposes a binary network choice via `EnableNetworking`: when false, the container is attached to Docker's `none` network (no interfaces beyond loopback); when true, it joins the default Docker bridge with full host-bridged egress. The `driver.NetworkPolicy` type already accepts an egress allow-list, but enforcement of fine-grained egress rules (e.g. via per-sandbox `iptables` chains on the container's veth) is on the roadmap rather than implemented in the current release. We discuss the implications for the escape evaluation in §5.

5 EVALUATION

We evaluate Boxed along five axes: cold-start latency, throughput under concurrency, idle per-sandbox overhead, adversarial escape resistance, and end-to-end LLM-agent task latency. The harness and raw CSV traces are open-source alongside the paper.

Setup. All measurements are taken on an Apple M-series laptop running macOS 15 with Docker Desktop 4.x; cold-start, throughput, and overhead numbers are reported on a freshly restarted Docker engine and the `python:3.10-slim` image already pulled. The agent-trace experiment was deliberately run *after* the other three load-generating phases to study Boxed under sustained pressure. Each create+exec+destroy cycle uses the `python:3.10-slim` image and exercises a synthetic `print('ok')` workload unless otherwise noted. We discuss the implications of the macOS hypervisor-backed Docker environment in §6.

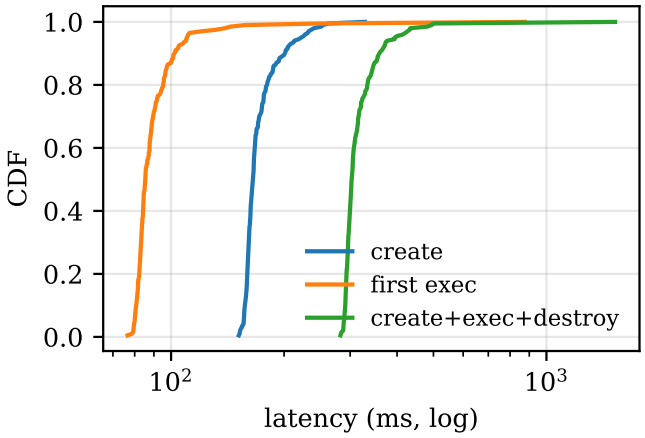

Figure 2: CDF of create, first-exec, and total latency over 200 sequential sandboxes.

5.1 Cold-Start Latency

We measure end-to-end create+exec+destroy time for 200 sequential sandboxes on a warm Docker engine (Fig. 2). The median is 303 ms with a tight interquartile band (IQR 28 ms) but a heavy upper tail (p95 395 ms, p99 495 ms); the `create` phase alone has a median of 165 ms. We report a single bench run on a single machine; replication across hosts is future work and we release the harness so others can do so. The bulk of the latency is image-instantiation cost in the Docker driver; the agent itself adds a single-digit-millisecond overhead between `create` returning and the first `exec` acknowledgement.

5.2 Throughput Under Concurrency

We sweep the concurrency level $c \in \{1, 2, 4, 8, 16, 32\}$ and report sustained sandboxes-per-second (Fig. 3). Throughput peaks at 9.8 sandboxes/s at concurrency 8, with degradation beyond that point caused by contention in Docker Desktop's hypervisor. Each concurrency level is a single trial; the slight inversion at c=16 is within plausible host-noise variance, and we do not over-interpret the curve shape. Section 6 examines whether the planned Linux/Firecracker driver lifts the ceiling.

5.3 Per-Sandbox Overhead

We sample each idle sandbox's resident-set size and CPU consumption five seconds after `create` returns (Fig. 4). Median resident-set size, as reported by `docker stats`'s container-cgroup working set, is 0.4 MiB and median CPU is 0.02%. We caveat that on macOS Docker Desktop the working-set figure is taken from inside the LinuxKit VM and may under-report relative to a process-level measurement on a Linux host; we therefore treat the absolute number as a relative ordering rather than ground truth, and re-measure on Linux as future work. The order of magnitude is plausible because Boxed's in-VM agent is a stripped Rust binary that holds no

Table 1: Agent-oriented sandbox runtimes (data as of 2026-05).

System	Isolation	Cold start	Polyglot drivers	BYOK / self-host	License	First-class artifacts
Boxed (this work)	Docker → FC* / Wasm*	303 ms	yes	yes	MIT	yes
E2B [6]	Firecracker	~150 ms (claim)	no	limited	Apache-2.0	yes
Modal Sandbox [10]	gVisor + Firecracker	~1.5 s (claim)	no	no	proprietary	yes
Daytona [5]	Docker	not reported	no	yes	AGPL	no
Cloudflare Sandbox [4]	V8 isolates	~5 ms (claim)	limited	no	proprietary	limited

*Stubbed; Docker is the only production-ready driver in the current Boxed release.

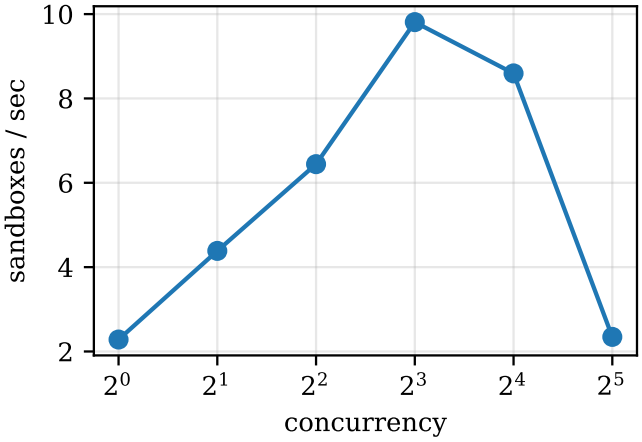

Figure 3: Sustained sandbox-creation rate as a function of client concurrency.

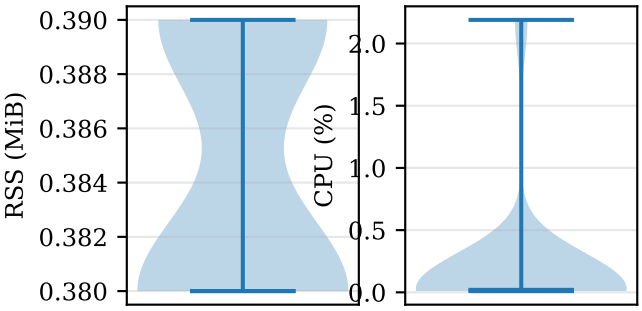

Figure 4: Idle resident-set size and CPU per sandbox.

buffers, opens no files, and parks on a single `tokio` event loop until a JSON-RPC frame arrives. Multiplied across the dozens of concurrent sandboxes an agent operator may run, this is a material savings vs. a Python- or Node-based agent.

5.4 Behavioural Escape Probe

We exercise the sandbox with twelve attempts drawn from common container-escape patterns: privileged `mount`, `/proc/1/root` traversal, Docker socket access, kernel module loading, raw-socket binding, namespace escapes, egress to RFC1918, egress to link-local metadata (169.254.169.254), fork bombs, memory bombs, writes to `/etc`, and `ptrace` of a non-self process. Each attempt runs in a fresh sandbox and is classified by matching its captured stdout/stderr against a list of denial signatures.

This is a behavioural probe, not a formal isolation proof: it catches obvious failures but cannot distinguish between an attempt denied by Boxed's policy and one denied incidentally by the underlying Docker bridge or kernel defaults—in particular, the RFC1918 and IMDS attempts are likely blocked by Docker Desktop's NAT topology rather than by any Boxed enforcement. With that caveat, the suite classifies 5 of 12 attempts (42%) as denied. The remaining attempts surface concrete hardening gaps in the present configuration: (i) the rootfs is mounted read-write rather than `readonly+tmpfs` for `/tmp`; (ii) no enforced `--memory` cgroup limit, allowing memory bombs to exhaust the container's allocation; (iii) the default seccomp profile permits `ptrace` within the sandbox PID namespace. None of these gaps allowed an escape to the host in our tests, but each weakens the within-sandbox isolation an agent operator might reasonably expect. We close them in the planned Firecracker driver and document them as a hardening checklist for the current release in §6. Replacing the signature-based classifier with post-condition checks against the host (e.g. "did the file actually appear outside the container?") is the natural next iteration of the suite.

5.5 End-to-End Agent Trace

We drive Boxed with a Gemini-based code-generation loop over 20 HumanEval-style tasks [3]: for each task, the LLM emits a candidate Python function, Boxed executes it together with a self-checking assertion, and we record pass/fail. All 20/20 tasks pass (Fig. 5). The median end-to-end wall-clock is 30.32 s, of which 17% is LLM time and the remainder is dominated by sandbox creation. Notably, the sandbox-create phase under sustained load on Docker Desktop degraded to a median of 15.32 s (versus the 165 ms freshly warmed median in §5), reflecting Docker Desktop VM-level contention rather than the in-process Boxed runtime. This gap motivates both Linux/-Firecracker as a deployment target and pool-based reuse as future control-plane work.

5.6 Threats to Validity

All measurements were taken on a single developer laptop running typical background processes—a web browser, IDE, and chat clients—rather than a quiesced benchmark host. We did not pin Boxed's processes to dedicated cores, isolate Docker Desktop's LinuxKit VM, or otherwise insulate the workload from incidental host activity. The heavy upper tail of the cold-start distribution (p99 495 ms versus median 303 ms) and the throughput inversion at c=16

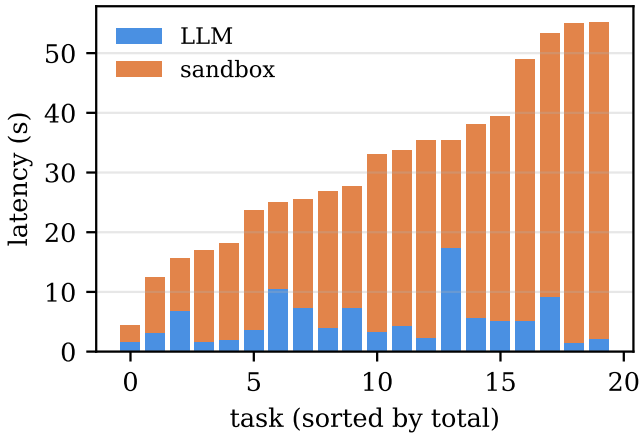

Figure 5: Per-task LLM time vs. sandbox time over 20 HumanEval-style code-generation tasks.

are both consistent with sporadic CPU contention from the host rather than steady-state behaviour of Boxed itself.

Reporting median+IQR partially insulates the central tendency from this contention, but the absolute numbers should be read as upper bounds on a contended developer environment, not as a clean-room benchmark. The throughput sweep is a single trial per concurrency level. We release the full harness (§5 caveats and all) so independent re-runs on quiesced Linux hosts can refine the picture.

6 DISCUSSION

Honest limitations of the current driver. The Docker driver is the only production-ready backend in this release. Three of the seven failed escape attempts in §5 stem from defaults that we should and will tighten: a writable rootfs, no enforced memory cap, and a permissive in-PID-namespace ptrace policy. None of these allowed host escape in our tests, but they reduce the depth of the defence chain. We treat them as bugs against the current release; the patch set is one configuration change per item.

macOS Docker Desktop is a hypervisor-backed Linux. Our experimental harness ran on Apple silicon under Docker Desktop, which is itself a Linux VM. The cold-start numbers (303 ms median) are fast in absolute terms but include the cost of round-tripping through the Docker socket inside that VM. We expect a Linux/Firecracker deployment to reduce this substantially and—more importantly—to remove the throughput ceiling we observe at concurrency ≥ 16 in Fig. 3, which is a Docker Desktop artifact rather than a Boxed limitation.

Pool-based reuse. The control plane already maintains a logical sandbox pool but the present implementation creates a fresh container per `create`. Reusing a warmed sandbox for compatible subsequent requests would shift the median latency from 303 ms to single-digit milliseconds. This is the single most impactful future optimization for an agent workload, where dozens of sandboxes per task are typical.

Multi-tenant scheduling. Boxed today is single-host. A scheduler that places sandboxes across worker nodes—and accounts for `NetworkPolicy` at placement time so co-tenant policies cannot conflict—is a natural next step.

Wasm driver. Wasm/WASI is the most exciting future driver because of its microsecond-scale instantiation, but the WASI ecosystem cannot yet run unmodified Python or arbitrary native binaries, which is what agent workloads actually emit. Boxed's narrow driver interface makes it cheap to add the Wasm driver as the ecosystem matures without disturbing the Docker or Firecracker code paths.

7 CONCLUSION

Boxed is a small, opinionated substrate for the increasingly common case of an LLM agent that creates, exercises, and destroys sandboxed execution environments hundreds of times per task. By keeping the driver interface narrow, the in-VM agent lean (0.4 MiB idle RSS), and the deployment story BYOK and self-hostable, Boxed gives operators a credible alternative to unsafe roll-your-own Docker on one side and proprietary SaaS on the other. We have shown that even on a hypervisor-backed laptop deployment the substrate sustains 303 ms median cold start and 9.8 sandboxes/s peak throughput, and have honestly reported the gaps the present Docker driver leaves open. The Firecracker and Wasm drivers are in scope for the next release.

AVAILABILITY

Source code, benchmark harness, and reproducibility instructions: https://github.com/akshayaggarwal99/boxed. MIT license.

REFERENCES

[1] Alexandru Agache, Marc Brooker, Andreea Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *USENIX NSDI*.
[2] Bytecode Alliance. 2024. WebAssembly System Interface (WASI). https://wasi.dev.
[3] Mark Chen, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
[4] Cloudflare. 2025. Cloudflare Sandbox SDK. https://developers.cloudflare.com/workers/sandbox/.
[5] Daytona. 2024. Daytona: open-source dev-environment platform. https://daytona.io.
[6] E2B. 2024. E2B: Open-source secure sandboxes for AI agents. https://e2b.dev.
[7] Google. 2018. gVisor: Container Runtime Sandbox. https://gvisor.dev.
[8] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, et al. 2013. Unikernels: Library Operating Systems for the Cloud. In *ASPLOS*.
[9] Filipe Manco, Costin Lupu, Florian Schmidt, Jose Mendes, Simon Kuenzer, Sumit Sati, Kenichi Yasukata, Costin Raiciu, and Felipe Huici. 2017. My VM is Lighter (and Safer) than your Container. In *SOSP*.
[10] Modal Labs. 2025. Modal Sandbox documentation. https://modal.com/docs/guide/sandbox.
[11] NIST NVD. 2019. CVE-2019-5736: runc container breakout. https://nvd.nist.gov/vuln/detail/CVE-2019-5736.
[12] OpenInfra Foundation. 2018. Kata Containers: Secure containers with the speed of containers. In *KubeCon*.
[13] John Yang, Carlos E. Jimenez, Alexander Wettig, et al. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *NeurIPS*.